Release Notes for Neuron Component: NKI Library

Contents

- NKI Library (NKI-Lib) (Neuron 2.29.0 Release)
- NKI Library (NKI-Lib) (Neuron 2.28.0 Release)
- NKI Library (NKI-Lib) (Neuron 2.27.0 Release)

This document is relevant for: `Inf1`, `Inf2`, `Trn1`, `Trn2`, `Trn3`

The release notes for the NKI Library Neuron component. Read them for the details about the changes, improvements, and bug fixes for all release versions of the AWS Neuron SDK.

NKI Library (NKI-Lib) (Neuron 2.29.0 Release)

Date of Release: 04/09/2026

What's New

This release promotes `find_nonzero_indices` from experimental to a core subkernel and adds 7 new experimental kernels (Conv1D, Transformer TKG, 3 collective communication kernels, Top-K Reduce, and Dynamic Elementwise Add). Existing kernels receive sequence packing support, MXFP quantization paths, and expanded dimension limits. PyTorch reference implementations are added for 22 kernels.

New Core Additions

- find_nonzero_indices (promoted from experimental) — Finds indices of nonzero elements along the T dimension using GpSimd `nonzero_with_count` ISA. Optimized for LNC2 sharding. Supports token counts up to 65536 and column counts up to 128.

v2.29.0

Skip to main content

New Experimental Kernels

- Conv1D — 1D convolution using tensor engine with replication strategy. Supports stride, padding, dilation, optional bias, activation fusion, and LNC sharding.
- Transformer TKG — Multi-layer transformer forward pass megakernel for token generation. Executes attention block, all-reduce, MLP, and residual connections across a configurable number of layers.
- Fine-Grained All-Gather — Ring-based all-gather for TRN2 using collective permute with double buffering to overlap communication and data movement.
- FGCC (All-Gather + Matmul) — Fused all-gather and matrix multiplication for TRN2, overlapping communication with compute.
- SBUF-to-SBUF All-Gather — Two variants: `allgather_sb2sb` for small tensors fitting in SBUF and `allgather_sb2sb_tiled` with tiling and LNC support for larger tensors.
- Top-K Reduce — Gathers scattered rows by packed global token index and reduces along the K dimension for MoE output. Supports LNC sharding on the hidden dimension.
- Dynamic Elementwise Add — Elementwise addition with runtime-variable M-dimension tiling using dynamic loop bounds.

Improvements

- Attention CTE Kernel: Added `mm_out_dtype` parameter for controlling matmul output dtype. Added `bound_min` / `bound_max` parameters for sequence packing support (per-query KV range bounds). Increased max batch size from 32 to 512. Increased max sequence length from 36864 to 131072.
- Attention BWD Kernel: Added `bound_min` / `bound_max` parameters for sequence packing support. Added support for large batch size.
- Attention TKG Kernel: Added `start_pos_ids` parameter for explicit KV cache position control to support sliding window masking.
- Attention Block TKG Kernel: Added `rmsnorm_QK_pre_rope_W_Q` / `rmsnorm_QK_pre_rope_W_K` parameters for fused QK-norm before RoPE. Added KVDP attention sharding support (`KVDP`, `KVDP_replica_group`). Added `enable_fa_s_prior_tiling` for overriding flash attention s_prior tiling.
- MLP Kernel: Added `sbm` (BufferManager) parameter for custom SBUF memory management. Added MXFP4/MXFP8 quantization path.
- MoE TKG Kernel: Added new dynamic all-expert algorithm that uses `bloc` v2.29.0 `is_all_expert_dynamic` args. Expanded support for small I and added support for sharding

Skip to main content

- Output Projection CTE Kernel: Added `output_dtype` parameter for controlling output data type.
- Output Projection TKG Kernel: Added `sbm` (BufferManager) parameter for custom SBUF memory management.
- QKV Kernel: Added `is_h_dim_4h_transposed` and `weight_layout` parameters for flexible weight layout support.
- **rmsnorm_tkg / layernorm_tkg**: Added `shard_on_h` parameter for sharding on the hidden dimension.
- Added PyTorch reference implementations for 22 kernels for testing and validation.

Breaking Changes

- Router Top-K Kernel: The `output_in_sbuf`, `x_input_in_sbuf`, and `expert_affin_in_sb` parameters have been removed. The kernel now auto-detects SBUF inputs from the tensor buffer type. Callers passing these keyword arguments must remove them.
- QKV Kernel: The `is_input_swizzled` parameter has been removed and replaced by `is_h_dim_4h_transposed` (same position, same default `False`) and a new `weight_layout` parameter. Callers using `is_input_swizzled` by name must rename to `is_h_dim_4h_transposed`.
- QKV Kernel (TKG variant): New parameter `is_h_dim_4h_transposed` has been inserted after `quantization_type`. Callers using positional arguments for `qkv_w_scale` or later parameters must update to use keyword arguments.
- Attention CTE Kernel: New parameter `mm_out_dtype` has been inserted between `softmax_dtype` and `cp_offset`. Callers using positional arguments for `cp_offset`, `global_cp_deg`, or `cp_strided_q_slicing` must update to use keyword arguments.
- Attention TKG Kernel: New parameter `start_pos_ids` has been inserted after `rope_pos_ids`. Callers using positional arguments beyond `rope_pos_ids` must update to use keyword arguments.
- Attention BWD Kernel: New parameters `bound_min` and `bound_max` have been inserted between `sinks_ref` and `use_causal_mask`. Callers using positional arguments for `use_causal_mask` or later parameters must update to use keyword arguments.
- Attention Block TKG Kernel: The keyword-only marker (`*`) has been removed and multiple parameters have been reordered. New pre-RoPE QK-norm parameters (`rmsnorm_QK_pre_rope_W_Q`, `rmsnorm_QK_pre_rope_W_K`) have been ad
  `softmax_scale`, `k_scale`, and `v_scale` have been moved to optional p

- **rmsnorm_tkg / layernorm_tkg**: New parameter `shard_on_h` has been inserted before `use_heap_memory` and `sbm`. Callers using positional arguments beyond `single_core_forced` (rmsnorm) or `eps` (layernorm) must update to use keyword arguments. Helper functions `process_rmsnorm_tile`, `rmsnorm_tkg_llama_impl`, and `layernorm_tkg_llama_impl` have been made private (prefixed with `_`).
- **SbufManager** has been renamed to **BufferManager**. A backward-compatible alias `SbufManager = BufferManager` is provided, so existing code using `SbufManager` will continue to work.
- MoE TKG: Replaced boolean sharding flags (`shard_on_I`, `shard_on_T`) with `LNCShardingStrategy` enum in down projection interfaces.
- MoE TKG MX quantization files restructured: `down_projection_mx_shard_I.py` and `gate_up_projection_mx_shard_I.py` replaced with `all_expert_mx_utils.py`, `down_projection_mx.py`, and `gate_up_projection_mx.py`. Callers importing from the old file paths must update their imports.
- `find_nonzero_indices` has been moved from `nkilib.experimental.subkernels` to `nkilib.core.subkernels`. A backward-compatible re-export is provided, so imports via the experimental path continue to work.
- Removed usage of `nki.language.par_dim` throughout the library.

Bug Fixes

- Fixed MLP CTE indexing in gate proj row scales.
- Fixed QKV TKG `sb2sb_wrapper_kernel` signature missing QK-norm parameters.
- Fixed MLP failure for FP4 quantization with specific dimension combinations (`vnc=2, h=3072, i=384`).
- Fixed `bwmm_shard_on_H` with explicit TensorCopy from PSUM to SBUF for NKI Beta 3 compatibility.

Known Issues

NKI Library (NKI-Lib) (Neuron 2.28.0 Release)

What's New

v2.29.0

This release expands the NKI Library with 9 new kernels, bringing the total to 1

Skip to main content

experimental kernels include Attention Block TKG (fused attention block for token generation), Cross Entropy (forward and backward passes), Depthwise Conv1D, and Blockwise MM Backward for MoE training.

Existing kernels receive FP8 and MX quantization support across QKV, MLP, and both Output Projection kernels. Kernel utilities gain new TensorView methods, SbufManager logging improvements with tree-formatted allocation tracing, and new utilities including `interleave_copy`, `LncSubscriptable`, and `rmsnorm_mx_quantize_tkg`. Note that several breaking changes affect kernel signatures and utility APIs — see the Breaking Changes section for details.

New Core Kernels

- RoPE Kernel — Applies Rotary Position Embedding to input embeddings with optional LNC sharding and flexible layout support (contiguous and interleaved).
- Router Top-K Kernel — Computes router logits and top-K expert selection for Mixture of Experts models, with support for multiple layout configurations and sharding strategies.
- MoE CTE Kernel — Implements Mixture of Experts optimized for Context Encoding with multiple sharding strategies (block sharding, intermediate dimension sharding) and MxFP4/MxFP8 quantization.
- MoE TKG Kernel — Implements Mixture of Experts optimized for Token Generation with all-expert and selective-expert modes, supporting FP8 and MxFP4 quantization.
- Cumsum Kernel — Computes cumulative sum along the last dimension, optimized for batch sizes up to 2048.

New Experimental Kernels

- Attention Block TKG Kernel — Fused attention block for Token Generation that combines RMSNorm, QKV projection, RoPE, attention, and output projection in SBUF to minimize HBM traffic.
- Cross Entropy Kernel — Memory-efficient cross entropy loss forward and backward passes for large vocabularies using online log-sum-exp algorithm, optimized for LNC2.
- Depthwise Conv1D Kernel — Depthwise 1D convolution using implicit GEMM algorithm with support for arbitrary stride and padding values, optimized for TRN2.
- Blockwise MM Backward Kernel — Backward pass for blockwise matrix multiplication in MoE layers, computing gradients for all parameters with support for dropless M

Improvements

- QKV Kernel: Added FP8 quantization support (`quantization_type`, `qkv_w_scale`, `qkv_in_scale`), fused FP8 KV cache quantization (`k_cache`, `v_cache`, `k_scale`, `v_scale`, `fp8_max`, `fp8_min`, `kv_dtype`), block-based KV cache layout (`use_block_kv`, `block_size`, `slot_mapping`), and MX quantization input swizzling (`is_input_swizzled`).
- MLP Kernel: Added FP8 quantization support (`quantization_type`, `gate_w_scale`, `up_w_scale`, `down_w_scale`, `gate_up_in_scale`, `down_in_scale`, `quant_clipping_bound`), gate/up projection clamping (`gate_clamp_upper_limit`, `gate_clamp_lower_limit`, `up_clamp_upper_limit`, `up_clamp_lower_limit`), `skip_gate_proj` option, and fp16 support for TKG mode.
- Output Projection CTE Kernel: Added FP8 quantization support (`quantization_type`, `input_scales`, `weight_scales`).
- Output Projection TKG Kernel: Added FP8 quantization support (`quantization_type`, `weight_scale`, `input_scale`) and removed 512 restriction on non-transpose path.
- Attention CTE Kernel: Added strided Q slicing for context parallelism (`cp_strided_q_slicing`).
- RMSNorm-Quant Kernel: Added input dequantization scale support (`input_dequant_scale`).

Kernel Utilities

See Kernel Utilities Reference for full documentation.

- TensorView: Added `rearrange` method for flexible dimension reordering, `has_dynamic_access` for checking whether a view requires runtime-dependent addressing, and `key_in_dict` helper. The `slice` method now clamps the end index to dimension bounds instead of asserting.
- TiledRange: `TiledRangeIterator` now exposes an `end_offset` attribute, enabling kernels to determine the end position of each tile without manual calculation.
- SbufManager (Allocator): Added `get_total_space` and `get_used_space` for querying SBUF utilization, `set_name_prefix` / `get_name_prefix` for scoped naming, and `flush_logs` to emit buffered allocation logs. SbufManager now uses `TreeLogger` to provide hierarchical, tree-formatted logs of SBUF allocation and deallocation events, making it easier to debug memory usage across nested scopes.
- **QuantizationType**: Added `MX` enum value for microscaling quantization (MxFP4/MxFP8)
- **common_types**: Added `GateUpDim` enum for distinguishing gate vs up pro

- **rmsnorm_tkg / layernorm_tkg**: Both subkernels now accept a `TensorView` or `nl.ndarray` for input and require an explicit `output` tensor parameter, giving callers control over output placement.
- **New utilities**: Added `rmsnorm_mx_quantize_tkg` subkernel for fused RMSNorm with MX quantization in token generation, `interleave_copy` for interleaved tensor copy operations, `LncSubscriptable` for LNC-aware data access patterns, and `TreeLogger` for hierarchical allocation logging.

Breaking Changes

- The open source repository source directory has been renamed from `nkilib_standalone` to `nkilib_src`.
- MLP Kernel: The function has been renamed from `mlp_kernel` to `mlp`. New parameters have been inserted in the middle of the signature; callers using positional arguments beyond `normalization_type` must update to use keyword arguments.
- QKV Kernel: New parameters (`quantization_type`, `qkv_w_scale`, `qkv_in_scale`) have been inserted after `bias`; callers using positional arguments beyond `bias` must update to use keyword arguments.
- Output Projection TKG Kernel: The `bias` parameter is now optional (default `None`). New parameters (`quantization_type`, `weight_scale`, `input_scale`) have been inserted before `TRANSPOSE_OUT`; callers using positional arguments beyond `bias` must update to use keyword arguments.
- **TiledRangeIterator**: The constructor now requires a fourth positional argument `end_offset`.
- **TensorView**: The `sizes` attribute has been renamed to `shape`.
- **rmsnorm_tkg**: The `inp` parameter has been renamed to `input`. A new required `output` parameter has been added as the third argument. The `output_in_sbuf` parameter has been removed. New parameters `hidden_dim_tp` and `single_core_forced` have been added.
- **layernorm_tkg**: The `inp` parameter has been renamed to `input`. A new required `output` parameter has been added as the third argument. The `output_in_sbuf` parameter has been removed.

Bug Fixes

- Fixed attention TKG compilation and non-determinism issues.
- Fixed incorrect v_active slice indices in attention TKG block KV path.

- Fixed expert_affinities masking when `mask_unselected_experts` is True in MoE TKG.
- Fixed expert_index shape mismatch in MoE TKG for T > 128.
- Fixed MoE affinity mask handling for T not divisible by 128.
- Fixed MoE TKG MX weight generation x4 pack size.
- Fixed MLP CTE `force_cte_mode` parameter validation.
- Fixed output projection CTE mixed precision support.
- Fixed output projection TKG variable name typo.
- Fixed router_topk bias shape to satisfy NKI check requirements.
- Fixed tail iteration bug for sequences not a multiple of 128 in MoE CTE.
- Fixed reading extra partitions for last rank in MoE CTE.

Known Issues

NKI Library (NKI-Lib) (Neuron 2.27.0 Release)

What's New

This release introduces the NKI Library, which provides pre-built kernels you can use to optimize the performance of your models. The NKI Library offers ready-to-use, pre-optimized kernels that leverage the full capabilities of AWS Trainium hardware.

NKI Library kernels are published in the NKI Library GitHub repository. In Neuron 2.27, these kernels are also shipped as part of neuronx-cc under the `nkilib.*` namespace.

Accessing NKI Library Kernels

You can access NKI Library kernels in two ways:

- **Shipped version**: Import from the `nkilib.*` namespace (included with neuronx-cc in Neuron 2.27)
- **Open source repository**: Clone and use kernels from the GitHub repository under the `nkilib_standalone.nkilib.*` namespace

New Kernels

This release includes the following pre-optimized kernels:

- **Attention CTE Kernel** — Implements attention with support for multiple variants and optimizations
- **Attention TKG Kernel** — Implements attention specifically optimized for token generation scenarios
- **MLP Kernel** — Implements a Multi-Layer Perceptron with optional normalization fusion and various optimizations
- **Output Projection CTE Kernel** — Computes the output projection operation optimized for Context Encoding use cases
- **Output Projection TKG Kernel** — Computes the output projection operation optimized for Token Generation use cases
- **QKV Kernel** — Performs Query-Key-Value projection with optional normalization fusion
- **RMSNorm-Quant Kernel** — Performs optional RMS normalization followed by quantization to fp8

NKI Library Kernel Migration to New nki.* Namespace in Neuron 2.28

Some NKI Library kernels currently use the legacy `neuronxcc.nki.*` namespace. Starting with Neuron 2.28, all NKI Library kernels will migrate to the new `nki.*` namespace.

The new `nki.*` namespace introduces changes to NKI APIs and language constructs. Customers using NKI Library kernels should review the migration guide for any required changes.

NKI Library Namespace Changes in Neuron 2.28

Starting with Neuron 2.28, the open source repository namespace will change from `nkilib_standalone.nkilib.*` to `nkilib.*`, providing a consistent namespace between the open source repository and the shipped version.

Customers who want to add or modify NKI Library kernels can build and install them to replace the default implementation without changing model imports.

This document is relevant for: `Inf1`, `Inf2`, `Trn1`, `Trn2`, `Trn3`